Lab 01
Topic: JavaScript
Marks: 100
Duration: 180 Minutes

Instructions:

- This lab is organized so that you get to do JavaScript by hand, and know a thing or two before entering the industry.
- This lab is **not** a group activity. Hence, copying is **not** allowed.
- You have to create a `lab_01.js` file, link it with an html file `lab_01.html` and use your browser's console to demonstrate your work.
- Once you are done with the lab, create a repository in your GitHub account `web_tech_labs`, create a branch named `lab/01`, and push your work in it, and do not push a zip file. My web crawler will fetch all of your labs tonight at 11:59 PM.
- You can use W3Schools, and online available knowledge resources to do this lab.
- I know that you will be tempted to use AI tools to do this lab. If you really want to learn a thing do not use AI, but if you must then do not use AI generated code right away. Understand the solution provided by AI first, and implement the solution by hand.
- Cooperate with the Teaching Assistants they are there for your help, as well as for grading your effort. The Google Sheet for your marks will be shared via email.
- If you have any concerns about the contents of the lab, or the TA, you can always log you concern on this form: https://forms.gle/DCqNhvbQag6VX9dj9

Tasks:

1) Create a function `addition(a, b)` that takes two numbers as arguments and returns their sum. (10 marks)

2) Complete the three functions for bitwise AND, OR, and XOR. Each function takes two decimal integers and returns its result as an ordinary decimal integer. (10 marks)
   - `bitwiseAND(a, b)`
   - `bitwiseOR(a, b)`
   - `bitwiseXOR(a, b)`

3) Create a function `addUp(a)` that takes a number as an argument. Add up all the numbers from 1 to the number you passed to the function. For example, if the input is 4 then your function should return 10 because 1 + 2 + 3 + 4 = 10. (10 marks)

4) Show how far a task has progressed using a ten-slot text bar. Create `progressBar(a)` to return a string containing filled slots ("#") followed by empty slots ("-"). Each filled slot represents a complete 10% of percent; a partly filled slot stays empty. e.g.,
   `progressBar(10) // "#---------"`
   The returned string always contains exactly ten characters, with no brackets or spaces. For example, both 40 and 49 have four filled slots. (10 marks)

5) Create a function `minMax(arr)` that takes an array of numbers and returns both the minimum and maximum numbers in an array. (10 marks)
   e.g., `minMax([1,2,3,4,5,6]) // [1, 6]`

6) Write a function named `calculateRepoStats(repos)` that takes an array of repository objects. Using the `Math` object combined with array methods, compute and return an object containing:

- `maxStars`: The highest star count
- `minStars`: The lowest star count
- `avgStars`: The average star count, rounded down to the nearest whole integer
- `varianceScore`: The absolute difference between the max and min stars

Sample array: `const repos = [ { name: 'repo-a', stars: 12 }, { name: 'repo-b', stars: 45 }, { name: 'repo-c', stars: 3 } ];`
Hint: use `.map` function in this task (15 marks)

7) An e-commerce checkout page calculates dynamic service fees based on cart totals, requiring precise mathematical rounding to avoid floating-point precision bugs (e.g., 0.1 + 0.2 = 0.30000000000000004). (15 marks)
Write a function named `calculateFinalPrice(basePrice, taxRate, discountPercentage)` that:
   - Applies the discount to the base price: `basePrice * (1 - discountPercentage / 100)`
   - Adds a variable tax rate.
   - Rounds the results after multiplication/division to 2 decimal places precisely.
   - Ceils calculated final "loyalty points earned," where every whole or partial amount spent earns 1 point rounded up.

8) (20 marks) When building a data dashboard or an e-commerce product catalog, you receive a massive array of items from an API. Instead of rendering them all at once, you need to "paginate" the results so the user views them in manageable chunks, complete with navigation metadata.
Write a function named `paginateData(items, pageNumber, pageSize)` that takes three arguments:
   - `items`: An array of data (e.g., search results or products).
   - `pageNumber`: The current page requested (starting at 1).
   - `pageSize`: The number of items to display per page (default to 5 if not provided).

   The function should return an object with the following structure:
   - `data`: The sliced subset of items belonging to the current page.
   - `currentPage`: The current page number.
   - `totalPages`: The total number of pages needed to display all items.
   - `hasNext`: A boolean (true/false) indicating if a next page exists.
   - `hasPrev`: A boolean (true/false) indicating if a previous page exists.

```
const allProducts = ['Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Desk', 'Chair', 'Webcam', 'Headphones'];

const result = paginateData(allProducts, 2, 3);
console.log(result);
/* Expected Output:
{
    data: ['Monitor', 'Desk', 'Chair'],
    currentPage: 2,
    totalPages: 3,
    hasNext: true,
    hasPrev: true
}
*/
```